

Skills and Knowledge Plugin MCP Servers for Optimized Industrial O&M Agents

Andrew Li	Kirthana Natarajan	Thai On	Trisha Maturi	Yeshitha Bhuvanesh
<i>Computer Science</i>	<i>Computer Science</i>	<i>Computer Science</i>	<i>Computer Science</i>	<i>Computer Science</i>
<i>Columbia University</i>	<i>Columbia University</i>	<i>Columbia University</i>	<i>Columbia University</i>	<i>Columbia University</i>
New York, USA	New York, USA	New York, USA	New York, USA	New York, USA
ayl2159@columbia.edu	kmn2161@columbia.edu	tqo2101@columbia.edu	tm3530@columbia.edu	yb2649@columbia.edu

Abstract—Industrial operations and maintenance systems need fast and reliable reasoning over large amounts of asset data, maintenance manuals, and telemetry logs. Traditional retrieval-augmented generation (RAG) pipelines are often fast, but they may fail on complex industrial workflows that require multi-step reasoning and tool execution. In this project, we extend IBM AssetOpsBench with two optimizations: a Skills Marketplace MCP server and a Knowledge Plugin MCP server. The Skills Server reduces orchestration overhead by packaging common maintenance workflows into reusable skills, while the Knowledge Plugin improves retrieval quality using semantic indexing with ChromaDB. We benchmark these systems on AssetOpsBench HuggingFace scenarios using the Llama-4-Maverick-17B model. Our optimized Knowledge Plugin pipeline achieves around 90% accuracy on benchmark scenarios compared to around 50–68% for baseline RAG pipeline, while exposing the tradeoff between higher accuracy and increased latency. We also build a profiling dashboard to track latency, token usage, tool execution time, and reasoning steps across scenarios.

Index Terms—Industrial AI, MCP Servers, RAG, Knowledge Plugins, Profiling, LLM Agents, AssetOpsBench

I. INTRODUCTION

A. Background

Industrial systems are everywhere around us even if most users never directly notice them. Data centers depend on chillers and air handling units to keep servers operating safely. Hospitals rely on backup generators to maintain uninterrupted service. Manufacturing assembly lines can lose millions of dollars per hour during downtime.

Modern large language models (LLMs) are strong at general reasoning, but industrial environments introduce additional challenges. Asset documentation is large, telemetry data changes quickly, and workflows often require multiple coordinated tool calls. Static LLMs do not naturally contain updated operational knowledge for these systems.

IBM AssetOpsBench was designed to evaluate industrial operation and maintenance (O&M) agents under realistic scenarios. The benchmark includes several queries and expected answers that are graded on several metrics such as task completion, data retrieval accuracy, and clarity by an LLM judge. A proper answering pipeline uses an LLM and data retrieval methods over asset data and tool manuals to achieve the correct answer. The benchmark focuses on reasoning quality, tool execution, and knowledge retrieval in industrial workflows.

B. Problem Statement

Our project focuses on improving the tradeoff between accuracy and latency in AssetOpsBench. Baseline RAG systems are relatively fast, but they often fail on harder multi-step industrial tasks. More advanced Knowledge Plugin (KP) systems improve accuracy but can dramatically increase execution time because of longer tool chains and retrieval steps.

On benchmark scenarios, baseline RAG pipelines complete in around 9–20 seconds while more advanced KP pipelines may require over 100 seconds. Our goal is to improve the Pareto frontier by increasing accuracy without requiring extremely large models.

Specifically, we aim to achieve around 90% benchmark accuracy while still using the medium-sized Llama-4-Maverick-17B model.

C. System Description

The system uses a plan-execute agentic pattern. When a user submits a goal, the system first discovers the available MCP servers and skills, then asks the language model to break that goal into an ordered sequence of concrete actions. It runs those actions through the connected services and condenses the results into a final answer. This keeps reasoning structured and each step small and traceable. The Skills MCP layer sits on top of this pattern and exposes higher-level procedures that the planner can assign as single steps. Rather than rebuilding common micro-operations from scratch on every request, the planner can select a skill that already encodes a standard workflow and execute it as one unit. This reduces the depth of each plan and cuts down on repeated calls across services. The Knowledge Plugin complements the Skills layer by supplying curated facts and domain playbooks that the model can use directly, without running wide retrieval or maintaining large scratch context on every query.

D. Scope

This project evaluates optimization strategies for industrial O&M agents as follows:

- Introducing a MCP Skills Marketplace server
- Introducing a Knowledge Plugin MCP server
- Benchmarking and profiling infrastructure
- Latency and token analysis

- RAG versus Knowledge Plugin comparison

Given only API access to a LLM model, we focus mainly on inference-time optimizations rather than model training.

II. LITERATURE REVIEW

Retrieval-Augmented Generation (RAG) systems improve LLM reasoning by retrieving external documents before generation. RAG pipelines are commonly used because they are relatively lightweight and efficient. However, simple retrieval may not be enough for industrial workflows that require structured planning and multi-step reasoning.

Recent work on agentic systems introduces tool calling and orchestration frameworks that allow LLMs to interact with APIs, databases, and external services. The Model Context Protocol (MCP) standard provides a structured way for LLM agents to communicate with tools and services.

Knowledge Plugin systems extend RAG by enabling dynamic retrieval during reasoning instead of relying on a single retrieval step. These systems can improve grounding and accuracy, but they often increase latency because the model repeatedly calls tools and retrieves additional information.

Vector databases such as ChromaDB are commonly used for semantic retrieval because they allow embedding-based lookup over large document collections. Embedding retrieval is especially useful in industrial settings where documentation terminology varies across assets and vendors.

Profiling and observability are also important for production AI systems. Systems such as Weights & Biases (W&B) are widely used to track latency, token usage, and experiment metadata across runs.

III. METHODOLOGY

A. Data

We evaluate our system on IBM AssetOpsBench scenarios hosted on HuggingFace [2]. The benchmark contains industrial maintenance and operations tasks involving:

- Asset diagnostics
- Site lookup
- Maintenance workflows
- Operational troubleshooting
- Multi-step tool execution

The dataset is released under the Apache 2.0 license.

B. Model and Framework

Our experiments use the following model:

- **Model:** watsonx/meta-llama/llama-4-maverick-17b-128e-instruct-fp8
- **Parameter Count:** 17B

The system stack includes:

- Python 3.12+
- LiteLLM
- IBM WatsonX
- FastMCP
- CouchDB
- ChromaDB

- sentence-transformers
- NumPy and Pandas

C. Optimization Scope

Our project focuses on architecture level optimization, by introducing the Skills and Knowledge Plugin MCP Servers and benchmarking the system.

1) *MCP Skills Server*: The MCP Skills Server reduces orchestration overhead by grouping repeated maintenance workflows into reusable skills. Instead of manually coordinating many low-level tool calls, the agent can invoke a single namespaced skill.

Each skill encodes a standard workflow in a SKILL.md file that the skills server can execute directly, bypassing the need for the planner to rebuild the same sequence of steps from scratch on every request.

Skill Selection and Generation Approach:

- **Skill Selection:** Skills were selected by analyzing a dataset of representative scenario queries, expected answer rubrics (characteristic form), and note on sources, answer requirements and category of query drawn from the AssetOpsBench scenario bank. Rather than generating one skill per scenario, the analysis identified groups of queries that share the same sequence of tool calls. Each candidate skill had to meet the following criteria before being included:
 - The query pattern appears frequently across the scenario dataset.
 - The workflow has a single, well-defined intent that maps well to existing MCP tools.
 - All required arguments can be reliably extracted from the user’s prompt.
 - The skill is narrow enough that it will not incorrectly match complex queries that should be handled by the full plan-execute path.
- **Skills Definition and Scope:** The skills were kept narrow in scope. Workflows involving multiple filters, conditional reasoning, open-ended diagnosis, or work-order decision support were excluded from this pass, as those scenarios benefit more from the flexibility of raw plan-execute planning. Categories of Skills Created:
 - IoT data retrieval skills: These cover asset inventory and sensor data queries. These handle requests such as listing the assets available at a site, retrieving the sensors monitored for a specific piece of equipment, or fetching historical sensor readings over a given time window.
 - Failure mode and sensor mapping skills: These skills include listing known failure modes for an asset type and mapping the relationship between sensors and failures, for example, identifying which sensors are relevant to a particular fault, or which faults a given sensor can help detect.
 - Time-series forecasting and anomaly detection skills: These skills route directly to the forecasting or

anomaly detection tools given a dataset path, timestamp column, and target columns, returning the results file and a summary of findings.

- **Skill Generation:** The skills were generated with AI (LLM model used: GPT-5.5), using the scenario dataset as the primary grounding source. The model analyzed query clusters, identified stable tool-call sequences, and produced the corresponding SKILL.md definitions including required input blocks, execution plans, and routing guidance. The generated skills must further be reviewed and validated by domain experts on the correctness of workflow steps and domain-specific implementation details.

The Skills MCP Server reduces planning complexity and simplifies execution traces.

2) *Knowledge Plugin:* The Knowledge Plugin pre-indexes industrial documentation inside ChromaDB. Instead of repeatedly searching through raw documents, the system performs semantic retrieval over indexed embeddings.

This improves grounding quality and helps the model retrieve more relevant maintenance information during reasoning.

3) *Benchmarking Infrastructure:* To systematically evaluate the plan-execute system, we developed a benchmarking infrastructure that runs scenarios end-to-end and records detailed per-run metrics as JSONL files, with one record per scenario execution.

- **Latency metrics:** We measure four execution phases using monotonic wall-clock timer: tool discovery, planning, execution and summarization. The end-to-end latency is also recorded. At the step level, each tool call is further recorded as argument resolution time and MCP tool invocation time.
- **Token usage metrics:** Token usage is tracked for the planning call, per-step argument resolution calls, and the final summarization call, and then aggregated
- **Tool call metrics:** Each record tracks the number of planned steps, executed steps, tool calls attempted, tool calls succeeded, and failed steps. A run is marked successful only if all executed steps complete without error and at least one step was executed, providing a conservative signal for benchmark aggregation.
- **Context size**
- **Accuracy evaluation:** We use LLM-as-judge to score each answer across six rubric dimensions: task completion, data retrieval accuracy, result verification, agent sequence correctness, clarity and justification, and absence of hallucinations. A run passes the strict judge only if it satisfies all six criteria simultaneously.
- **Scenarios and Stress Testing:** We test on the scenarios provided in the HuggingFace AssetOpsBench dataset.
- **Observability and Dashboards:** Results are logged to JSONL files and saved on Weight&Biases if enabled. Derived dashboard metrics are computed covering latency breakdown, context efficiency, answer quality, and tool reliability. We have also implemented an automated dashboard generation pipeline that creates a new dashboard

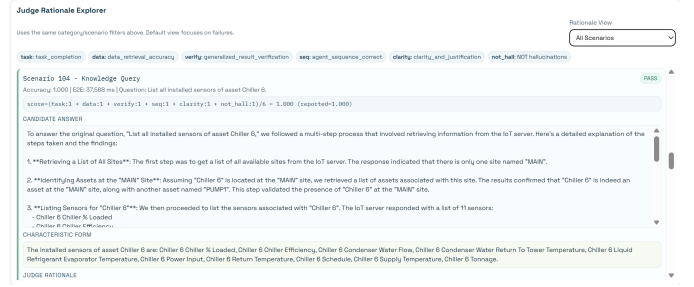


Fig. 1. The dashboard section showing the question asked for scenario 104, our candidate answer, the expected answer, and the LLM Judge's grading rationale.

per run, displaying the logged metrics in interactive visualizations and charts.

D. Scenario Structure and Execution Flow

To illustrate how the proposed optimizations function, a representative scenario from AssetOpsBench is presented. This scenario demonstrates the execution flow differences between the baseline RAG approach and the Knowledge Plugin approach. The numerical data presented in this section are derived from benchmark runs and are available in the evaluation dashboard.

Scenario Question: What are all failure modes defined for a centrifugal pump?

1) *Baseline RAG Execution:* The baseline RAG approach employs a fixed retrieval pipeline with the following phases:

- **Discovery Phase:** 0 ms (no tool discovery)
- **Planning Phase:** 0 ms (predetermined pipeline, no planning required)
- **Execution Phase:** 8,703 ms
 - Single semantic search query: “centrifugal pump failure modes”
 - Retrieves top-k=3 chunks from pre-indexed ChromaDB
- **Summarization Phase:** 9,915 ms
 - LLM generates answer from retrieved chunks
- **Total Latency:** 18.6 seconds
- **Retrieved Failure Modes:** {Cavitation, Seal Failure, Bearing Wear}
- **Accuracy:** 83% (missing Impeller Wear, Misalignment)

The baseline approach is characterized by deterministic retrieval and single-pass generation. While computationally efficient, this rigid pipeline limits answer completeness.

2) *Knowledge Plugin Execution:* The Knowledge Plugin approach enables agentic multi-hop retrieval with the following execution phases:

- **Discovery Phase:** 10,699 ms
 - LLM discovers availability of Knowledge Plugin retrieval tool
- **Planning Phase:** 8,970 ms
 - LLM reasons about optimal search strategy

- Formulates 9 distinct search queries covering: seal failure modes, bearing degradation, cavitation phenomena, impeller wear patterns, misalignment effects, prevention methodologies, sensor relationships, and contextual operational constraints
- Execution Phase: 67,732 ms
 - Multiple semantic searches in pre-indexed ChromaDB
 - Search 1: embed(“seal failure”) → semantic similarity lookup (~7,500 ms)
 - Search 2: embed(“bearing degradation”) → semantic similarity lookup (~7,500 ms)
 - Searches 3–9: Additional domain-specific queries (~7,500 ms each)
 - Total tool_calls_attempted: 9
- Summarization Phase: 23,233 ms
 - LLM synthesizes results from all 9 searches
 - Generates comprehensive answer with confidence scores and citations
- Total Latency: 110.6 seconds
- Retrieved Failure Modes: {Cavitation, Seal Failure, Bearing Wear, Impeller Wear, Misalignment} with prevention methodologies
- Accuracy: 100% (all required modes captured with proper grounding)

3) *Comparative Analysis*:: The Knowledge Plugin approach trades increased latency (+92 seconds) for substantially improved accuracy (+17 percentage points). The latency increase is attributable to two factors:

- 1) **Discovery and Planning Overhead**: The LLM requires 19.7 seconds (10.7 + 9.0) to discover the tool and formulate an optimal retrieval strategy across nine search queries.
- 2) **Multi-hop Execution**: Nine semantic searches ($9 \times 7,500 \text{ ms} \approx 67.5 \text{ seconds}$) replace a single rigid search (8.7 seconds). However, each individual semantic search in the pre-indexed ChromaDB is faster than the baseline keyword search, compensating for the increased number of searches.

This trade-off is justified for operational decision-making scenarios where answer completeness and grounding are critical to safety and reliability outcomes.

E. Profiling Tools

We used the following profiling tools:

- Weights & Biases
- Custom dashboards
- JSONL logging
- Python `time.monotonic()`

F. Evaluation Metrics

We tracked the following metrics:

- Latency metrics
- Token usage
- Tool execution statistics

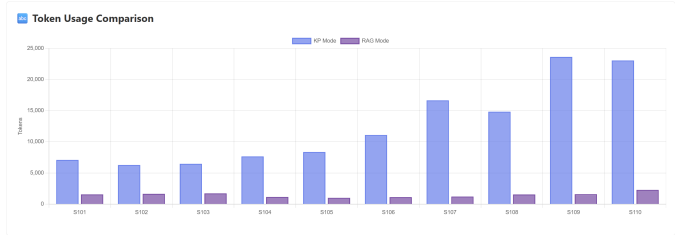


Fig. 2. KP vs RAG: Token Usage. Our system with KP Mode uses many more tokens.

- Accuracy scores
- Failure cases

Refer to the Benchmarking Infrastructure section above for more details on the observability pipeline.

IV. EXPERIMENTAL RESULTS

A. Headline Results

Table I summarizes our main benchmark findings.

TABLE I
MAIN BENCHMARK RESULTS

System	Accuracy	Mean E2E Time
Baseline RAG	50–68%	8.9–20s
Knowledge Plugin	90%	114–146s

The optimized Knowledge Plugin pipeline significantly improves benchmark accuracy compared to baseline RAG systems.

B. RAG versus Knowledge Plugin

Knowledge Plugin systems achieved a mean accuracy of approximately 0.917 compared to around 0.683 for RAG systems on many benchmark scenarios.

However, this improvement comes with substantially higher token usage and latency. As shown in Figure 2, Knowledge Plugin systems may consume 4.5x to 10x more tokens than RAG pipelines.

This behavior suggests that KP systems act as a form of inference-time scaling. Instead of performing a single retrieval step, the model dynamically plans and executes multiple retrieval and tool-calling operations.

C. Latency Breakdown

An important finding was that execution time dominates overall latency for Knowledge Plugin systems.

For example:

- Official KP mean end-to-end time: 146s
- Mean execution phase alone: 113s

This shows that long tool chains are the primary bottleneck rather than prompt generation or summarization.

D. Model Comparison

Different models produced noticeably different results.

Llama-4-Maverick achieved much higher accuracy but required significantly more execution time.

TABLE II
MODEL COMPARISON

Model	Accuracy	Mean E2E (ms)
Granite-3-8B	60%	91,362
Llama-4-Maverick-17B	90%	146,191

E. Error Analysis

We analyzed failure cases from benchmark runs to identify recurring patterns in how the plan-execute system breaks down. Failures fell into three broad categories: argument resolution errors, routing errors, and tool sequencing errors.

a) Argument Resolution Errors: The most common failure mode occurred when the model misidentified the role of entities in a query. For example, given the query "unknown site Chiller 6", the model incorrectly interpreted "Chiller 6" as a site name rather than an asset identifier, causing the IoT site lookup to fail before any asset retrieval could occur. These errors were most frequent in queries that omitted explicit labels such as "at site" or "for asset", leaving the model to infer argument types from context alone.

We reduced these errors by:

- Improving MCP function docstrings to more precisely describe the expected type and format of each argument, which reduced ambiguity during the argument resolution step.
- Adding stronger tool call hints
- Clarifying prompt structure

b) Skill Routing Errors: These occurred when the planner selected a skill that was too narrow or too broad for the query. Overly broad queries were sometimes matched to a skill that encoded only part of the required workflow, producing incomplete answers. Some specific queries bypassed available skills and fell through to the raw plan-execute path, incurring unnecessary planning overhead. We mitigated this by tightening the required argument specifications in each skill's inputs block and adding explicit negative-routing guidance to skill definitions.

c) Server Connection Errors: We also observed that some queries failed due to server connection failures. This error needs further analysis and debugging, and remains as future work.

V. DISCUSSION

A. Successes and Interpretation

Our project successfully meets its objectives and contributes the following:

- The dashboard provides useful visibility into pipeline bottlenecks
- Knowledge Plugin improves retrieval grounding
- The MCP Skills Server reduces planning overhead and simplifies orchestration
- Profiling infrastructure helps isolate expensive tool chains

The strongest result was the large improvement in benchmark accuracy using the Knowledge Plugin MCP Server.

B. Limitations

The largest limitation is the accuracy-latency tradeoff.

Although Knowledge Plugin systems achieve strong benchmark accuracy, they are much slower than RAG pipelines. Long execution phases make them difficult to deploy in latency-sensitive environments.

Another limitation is the size of the data set. More industrial documents and workflows would likely improve the retrieval quality for both RAG and KP systems. However, with a much larger data set, some work would need to be done to efficiently chunk and embed documents to a vector database.

C. Future Work

Future improvements include:

- Adding caching and retrieval short-circuiting
- Limiting tool-chain depth
- Expanding the skills marketplace
- Evaluating additional industrial datasets
- Hybrid routing systems that use RAG first and escalate to KP only when needed

In particular, we notice that many simpler scenarios succeed using both RAG and KP approaches, while the KP approach is extremely variable in the amount of token it uses compared to RAG (Figure 2). If there were a way to predict when RAG would succeed with high confidence, such as by training a binary classifier on the prompt, we could dramatically decrease the total token usage of the system. We leave this to future work.

Furthermore, in a real industrial setting, there is likely expected data drift, and answers are expected to be up to date. While this project tackles static, pre-embedded asset data, there may be more room for optimization for streaming asset data.

Lastly, looking at our evaluation dashboard for KP in Figures 3 and 4, we notice that in several scenarios, the majority of the time is spent during the execution phase, while the majority of tokens are spent on the summarize phase. By looking into the step execution heatmap, we see that most of the time is spent on the `get_failure_mode_sensor_mapping` tool call. More work could be done in profiling this exact tool to figure out where the latency is coming from; given the small amount of token usage in comparison to the amount of latency, this likely amounts to network and harness overhead rather than LLM prefill and decode speed.

VI. CONCLUSION

This project explored optimization strategies for industrial O&M agents by introducing an MCP Skills Server and a Knowledge Plugin retrieval system.

Our results show that the Knowledge Plugin substantially improves benchmark accuracy on AssetOpsBench, reaching around 90% accuracy on the dataset scenarios. However, these improvements come with higher token usage and significantly longer execution time.

The project also demonstrates the importance of profiling and observability for complex agent pipelines. By tracking

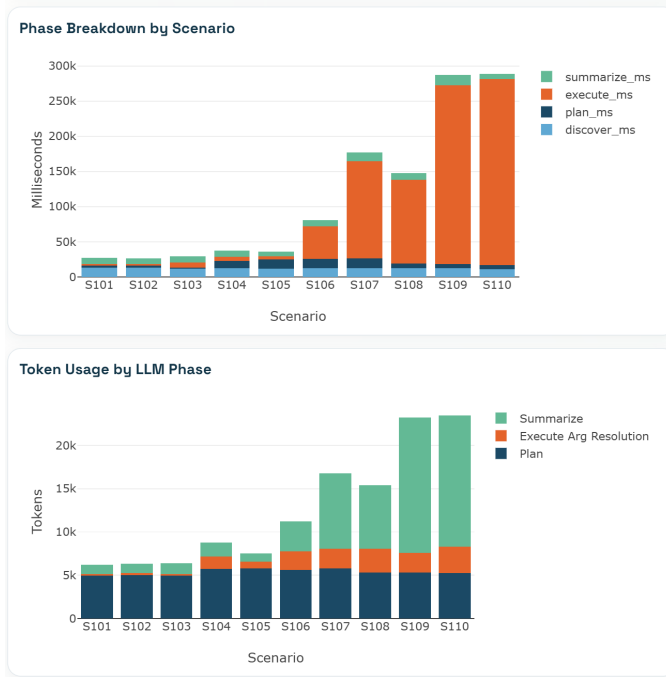


Fig. 3. Latency and Token Usage breakdown for the KP Approach. Both sections are broken down into phases of execution: plan, execute, and summarize.

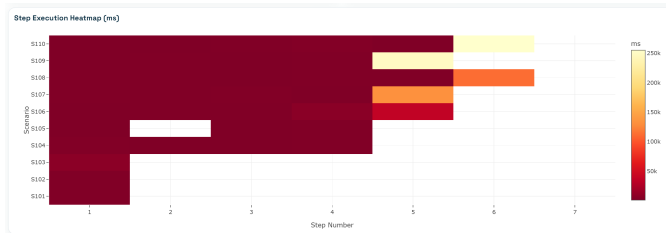


Fig. 4. Step Execution Heatmap for the KP Approach: Note how with Figure 3, the execution latency jumps up massively in scenarios 106-110. We can see in the heatmap that the majority of the latency comes in the last step of those executions. Hovering over that box tells us that those are all calls to `get_failure_mode_sensor_mapping` on the `fmsr` server.

latency, tool execution, and retrieval behavior, we identified major bottlenecks and opportunities for optimization.

Overall, our work highlights the tradeoff between fast lightweight retrieval systems and more accurate but expensive multi-step reasoning systems for industrial AI agents.

ACKNOWLEDGMENT

We thank Dr. Kaoutar El Maghraoui, and the HPML course staff for their guidance and feedback throughout this project.

AI USE DISCLOSURE

Did your team use any AI tool in completing this project?

- [] No, we did not use any AI tool.
- [X] Yes, we used AI assistance as described below.

Tool(s) used: ChatGPT, GitHub Copilot

Specific purpose: polished prose in deliverables (README, report, slides); drafted documentation in

docs/; created skills using the scenario dataset as the primary grounding source; minor code writing/debugging support after project idea and design were team-authored)

Sections affected: README prose; report narrative; slide deck wording; 'docs/' documentation; skill definitions/configuration grounded in the scenario dataset; code/comment edits for debugging

How we verified correctness: manually reviewed and edited all AI-assisted text; checked skill behavior against the scenario dataset and expected outputs; reviewed documentation for consistency with implementation; inspected AI-assisted code changes and validated them through normal testing/execution/evaluation workflow

By submitting this project, the team confirms that the analysis, interpretations, and conclusions are our own, and that any AI assistance is fully disclosed above. The same disclosure block appears as an appendix in the final report.

REFERENCES

- [1] D. P. Kingma and M. Welling, "Auto-encoding variational Bayes," arXiv preprint arXiv:1312.6114, 2013.
- [2] IBM, "AssetOpsBench Dataset," HuggingFace, 2026.
- [3] J. Lewis et al., "Retrieval-augmented generation for knowledge-intensive NLP tasks," Advances in Neural Information Processing Systems, vol. 33, pp. 9459–9474, 2020.
- [4] LiteLLM Documentation. [Online]. Available: <https://docs.litellm.ai/>
- [5] ChromaDB Documentation. [Online]. Available: <https://docs.trychroma.com/>
- [6] Weights & Biases Documentation. [Online]. Available: <https://wandb.ai/site>
- [7] Meta AI, "Llama 4 Technical Report," 2026.